

EduGenie

Google Gemini Powered Learning Assistant

Project Report: Documentation vs Source Code Comparison

Source documents compared	Original project document (PDF, Smartbridge / SmartInternz) and Edugenie-Ai-main.zip (source code)
Original submission	Varsha
Report prepared	October 2026
Method	Static review of every source file (backend, frontend, config, tests, Docker). The application was not run: the review environment has no network access or installed dependencies.

1. Summary

The source code is a cleaner, better-organised rewrite of the project described in the original document. All five features (Q&A, explanation, quiz, summary, learning path) exist, and the backend is more robust than the document describes: it has centralised configuration, a shared Gemini service, schema-validated quiz output, proper HTTP error codes, tests and Docker support.

However, the frontend and the document no longer match the code, and three small bugs in the JavaScript would stop the page from working at all. Key points:

- **Matches the document:** five modules, five endpoint paths, FastAPI + Jinja2 + HTML/CSS stack, Gemini for Q&A, quiz, summary and learning path, optional LaMini-Flan-T5 for explanations.
- **Changed on purpose (improvements):** new google-gemini SDK, app/ package layout, POST + JSON for every endpoint, Pydantic-validated quiz output, .env configuration.
- **Broken (critical):** app.js looks up element ids that do not exist in index.html, and the Learning Path option uses a different value in HTML and JS (section 4).
- **Missing versus the document:** interactive quiz with "Check Answer" feedback, and the learning-path timelines/resources promised in Scenario 3.
- **Model risk:** the document's gemini-1.5-pro is retired, and the model default in config.py could not be confirmed as a real model name.

Section 5 gives the exact fixes. Section 6 is the updated project documentation that reflects the code as it exists.

2. Architecture and Folder Structure

Request flow is unchanged from the original diagram: the user selects a task in the browser, the frontend posts to the matching FastAPI endpoint, the endpoint calls a service module, and the module calls Gemini (or the local LaMini model for explanations when enabled). The result is returned as JSON and displayed.

Original document (flat layout)

```
EduGenie/  
main.py  explanation_module.py  qna.py  quiz_module.py  
summary_module.py  learning_path.py  requirements.txt  
templates/index.html  static/style.css
```

Source code (package layout)

```
EduGenie/  
app/  
  main.py          FastAPI app, routes, error mapping  
  config.py        Settings via pydantic-settings (.env)  
  schemas/models.py Pydantic models (TextRequest, QuizResponse, ...)  
  services/  
    gemini_service.py Shared Gemini client, generate_text / generate_structured  
    qna.py explanation_module.py quiz_module.py  
    summary_module.py learning_path.py  
templates/index.html  static/style.css  static/app.js  
tests/test_api.py  
Dockerfile docker-compose.yml requirements.txt requirements-local.txt  
.env.example .gitignore README.me
```

3. Detailed Comparison: Document vs Code

Status key: **Match** = same as document; **Changed** = differs (usually an improvement); **New** = in code only; **Missing** = in document only.

Area	Original document	Source code	Status
Gemini SDK	google.generativeai, genai.configure(), GenerativeModel	google-genai (genai.Client), cached client created from the API key in .env	Changed
Gemini model	models/gemini-1.5-pro, hard-coded in each module	One setting GEMINI_MODEL. Default in config.py: gemini-3.8-flash; in .env.example: gemini-2.5-flash	Changed (see H1)
Folder layout	Flat files at project root	app/ package with services/ and schemas/; run as app.main:app	Changed
Explanation	LaMini-Flan-T5-783M always, loaded at import; sampling, temp 0.7, 150 tokens	Gemini by default. LaMini only if LOCAL_EXPLANATION=true; greedy decoding, 200 tokens; falls back to Gemini on failure. Structured 4-part prompt	Changed
Q&A	GET /qa?question=; raw question sent to Gemini; errors returned as text	POST /qa with {"text": ...}; prompt asks for Direct Answer / Explanation / Example	Changed
Quiz generation	Prompt asks for JSON; regex clean_json_block strips code fences; json.loads; answer is a letter ("A")	Gemini structured output with QuizResponse schema (exactly 3 questions, 4 options each); field is correct_answer (full option text); no regex cleanup needed	Changed
Quiz response key	{"quiz": [...]}	{"questions": [...]}	Changed
Summary	One-line prompt: summarise in simple language	Prompt with 5 requirements (keep key info, remove repetition, simple language, exam revision, bullets)	Changed
Learning path	GET /learn/recommendations?topic=; beginner to advanced, with resources (videos, articles, books)	POST with {"text"}; 7-section prompt (Basics to Final Project, Recommended Order). No timelines or resource list requested	Changed (see H3)
Endpoints	/qa, /explain/, /quiz, /summarize/, /learn/recommendations	Same five paths (no trailing slashes), all POST. Added /, /health, /api/info	Match + New
Error handling	Exceptions returned as strings inside a normal response	Errors mapped to JSON {"detail"} with 400 / 401 / 403 / 404 / 503 / 500	New
Configuration	API key in code	pydantic-settings reading .env; max input 20,000 chars; .env.example provided	New
System prompt	None	"EduGenie" tutor persona and 8 rules prepended to every request	New
Frontend layout	Five separate forms, each with its own result box	One page: task dropdown, one textarea, Generate and Clear buttons, one result box, hero section and feature cards	Changed
Quiz display	Radio options, Check Answer button, green/red feedback, shows correct answer	Quiz JSON printed as raw text (JSON.stringify)	Missing
Learning-path display	Raw markdown text in result box	Raw text in result box (markdown symbols such as ** and ## are shown as-is)	Match
Responsive CSS	Stated in document	Two @media blocks present in style.css	Match
Testing	Manual functional testing	tests/test_api.py: 3 tests (home page, /health, feature labels). No AI-endpoint tests	New

Area	Original document	Source code	Status
Run command	uvicorn main:app --reload	uvicorn app.main:app --reload; also Dockerfile (python:3.12-slim) and docker-compose	Changed + New
Dependencies	Python 3.10+, FastAPI, Uvicorn, Jinja2, Gemini key	Adds pydantic, pydantic-settings, python-dotenv, httpx, google-genai; local extras (transformers, torch, sentencepiece) in requirements-local.txt	Changed
README	Not part of document	README.me (misspelt extension), lists features and structure	New

4. Issues Found in the Source Code

Severity: **Critical** = the feature cannot work; **High** = wrong or incomplete versus the document; **Medium** = quality or maintainability risk.

ID	Severity	File	Problem	Effect
C1	Critical	static/app.js	JS reads <code>getElementById("generateBtn")</code> but the button in index.html is <code>id="submitBtn"</code> .	Returns null; <code>generateBtn.addEventListener</code> throws a <code>TypeError</code> , so the Generate button never works.
C2	Critical	static/app.js	JS reads <code>getElementById("error")</code> ; index.html has no such element (it has <code>id="status"</code>).	<code>errorBox</code> is null; the initial-state line at the bottom of the file and every error display throw <code>TypeError</code> s.
C3	Critical	app.js / index.html	Dropdown value is <code>learn</code> , but the JS switch handles <code>"learning"</code> .	Learning Path always ends in "Please select a valid task."
H1	High	config.py, .env.example	Default model <code>gemini-3.8-flash</code> could not be found in Google's published model and deprecation pages. <code>.env.example</code> uses <code>gemini-2.5-flash</code> , which Google lists for shutdown around 16 Oct 2026. The document's <code>gemini-1.5-pro</code> is already retired.	If <code>GEMINI_MODEL</code> is unset or invalid, every AI call returns the "model not found" (404) message.
H2	High	app.js	Quiz response is shown with <code>JSON.stringify</code> . No radio buttons, Check Answer, or correct-answer feedback.	Scenario 2 (quiz to test understanding) is not delivered in the UI.
H3	High	learning_path.py	Prompt has no request for timelines or learning resources. <code>LearningPathResponse</code> and <code>LearningStep</code> (with timeline and resources) are defined but never used.	Scenario 3 (levels, timelines, suggestions) only partly met; the SQL example in the document had time estimates and named resources.
M1	Medium	main.py	Routes take <code>request: dict</code> , so the <code>TextRequest</code> model (length and empty checks) and the response models are never used. A non-string text value fails with a generic 500.	Weak input validation; no OpenAPI schema for requests.
M2	Medium	explanation_module.py	With <code>LOCAL_EXPLANATION=true</code> , the LaMini pipeline is rebuilt on every request.	Very slow explanations (model reload each time).
M3	Medium	gemini_service.py	<code>clean_input</code> silently cuts the prompt at 20,000 characters, which can drop the end of a long prompt.	Long inputs may be truncated without telling the user.
M4	Medium	main.py	<code>error_response</code> checks for substrings such as "400" or "404" anywhere in the error text.	An unrelated message containing those digits could be mapped to the wrong status.
M5	Medium	tests/, README.me	Tests cover only static pages and <code>/health</code> . README is named <code>README.me</code> and debug in config is unused.	AI endpoints are untested; README will not render on GitHub.

5. Recommended Fixes

C1, C2 and C3: make the JavaScript match the HTML

```
// static/app.js - top of file
const generateBtn = document.getElementById("submitBtn"); // was "generateBtn"
const errorBox = document.getElementById("status"); // was "error"

// switch statement
case "learn": // was "learning"
    endpoint = "/learn/recommendations";
    break;
```

The existing code sets `style.display inline`, which overrides the `.hidden` class in `style.css`, so no further change is needed for visibility. Optionally add the error class to `#status` when showing an error: `style.css` already defines `.status.error` in red.

H1: set a valid model explicitly

Open Google AI Studio, pick a currently available model, and set it in `.env` (for example `GEMINI_MODEL=<model id from AI Studio>`). Then change the default in `config.py` to the same value so both files agree. Avoid models that Google has scheduled for shutdown.

H2: render the quiz interactively

In `showResult`, when `data.questions` exists, build one block per question with radio inputs and a Check Answer button, and compare the selected option text with `correct_answer`:

```
resultBox.innerHTML = "";
data.questions.forEach((q, i) => {
    const box = document.createElement("div");
    const title = document.createElement("p");
    title.textContent = "Q" + (i + 1) + ": " + q.question;
    box.appendChild(title);
    q.options.forEach(opt => {
        const label = document.createElement("label");
        const radio = document.createElement("input");
        radio.type = "radio"; radio.name = "q" + i; radio.value = opt;
        label.append(radio, " " + opt); // textContent-safe
        box.append(label, document.createElement("br"));
    });
    const btn = document.createElement("button");
    const fb = document.createElement("div");
    btn.textContent = "Check Answer";
    btn.onclick = () => {
        const sel = box.querySelector("input:checked");
        fb.textContent = !sel ? "Pick an option."
            : sel.value === q.correct_answer ? "Correct!"
            : "Incorrect. Correct answer: " + q.correct_answer;
    };
    box.append(btn, fb);
    resultBox.appendChild(box);
});
```

H3: ask for timelines and resources in the learning path

Add these lines to the prompt in `learning_path.py`: "For each level include an estimated time (for example 1-2 weeks) and 2-3 recommended resources such as courses, videos, or books." Alternatively, use `generate_structured(prompt, LearningPathResponse)` and render the steps in the UI.

M1 to M5

- Use body: `TextRequest` in each route and `response_model=...` for validated, documented APIs.

- Create the LaMini pipeline once (module-level variable or `lru_cache`).
- Return a 400 when input exceeds `max_input_chars` instead of truncating.
- Map errors using the Gemini SDK's exception status code rather than string search.
- Rename `README.me` to `README.md` and add tests that mock `generate_text` / `generate_structured` for each endpoint.

6. Updated Project Documentation (matches the code)

6.1 Project description

EduGenie is a lightweight AI-powered educational assistant built with FastAPI, Jinja2 and a plain HTML/CSS/JavaScript frontend. It uses Google Gemini through the `google-gemini` SDK for question answering, topic explanation, quiz generation, summarisation and learning-path recommendations. An optional local model, LaMini-Flan-T5-783M, can generate explanations on CPU when enabled.

6.2 API reference

Endpoint	Method	Request body	Success response	Module
<code>/</code>	GET	-	HTML page	<code>index.html</code>
<code>/health</code>	GET	-	<code>{"status": "ok", "service": "EduGenie"}</code>	<code>main.py</code>
<code>/api/info</code>	GET	-	Application name, version, model, feature list	<code>main.py</code>
<code>/qa</code>	POST	<code>{"text": "question"}</code>	<code>{"answer": "..."} </code>	<code>qna.py</code>
<code>/explain</code>	POST	<code>{"text": "topic"}</code>	<code>{"explanation": "..."} </code>	<code>explanation_module.py</code>
<code>/quiz</code>	POST	<code>{"text": "topic or passage"}</code>	<code>{"questions": [{question, options[4], correct_answer} x3]}</code>	<code>quiz_module.py</code>
<code>/summarize</code>	POST	<code>{"text": "passage"}</code>	<code>{"summary": "..."} </code>	<code>summary_module.py</code>
<code>/learn/recommendations</code>	POST	<code>{"text": "subject"}</code>	Learning path as text (string)	<code>learning_path.py</code>

Errors are returned as JSON `{"detail": "message"}` with status 400 (empty or rejected input), 401, 403, 404 (model not found), 503 (Gemini busy) or 500.

6.3 Setup and run

```
# 1. Create environment and install
python -m venv venv
source venv/bin/activate          # Windows: venv\Scripts\activate
pip install -r requirements.txt
# optional local explanation model:
pip install -r requirements-local.txt

# 2. Configure
cp .env.example .env
# GEMINI_API_KEY=<your key from Google AI Studio>
# GEMINI_MODEL=<a currently available Gemini model>
# LOCAL_EXPLANATION=false

# 3. Run (from the project root, because static/ and templates/ are relative)
uvicorn app.main:app --reload      # open http://127.0.0.1:8000

# 4. Tests
pip install pytest && pytest

# Docker alternative
docker compose up --build
```

6.4 Scenario coverage

Scenario	Backend	UI as delivered	After fixes in section 5
1. Ask "Which is the largest ocean?"	Works (POST /qa)	Blocked by C1/C2	Works
2. Generate quiz on Pythagoras Theorem	Works; returns 3 MCQs	Raw JSON only (H2); blocked by C1/C2	Interactive quiz
3. SQL learning path with timelines	Works, but no timelines/resources (H3)	Blocked by C1/C3	Works with prompt update

6.5 Future enhancements (from the original document)

Voice input, multilingual support, a mobile app with offline mode, progress dashboards, gamification (badges, streaks), adaptive learning paths, group study and teacher/parent dashboards, LMS integration (Moodle, Google Classroom), and image/PDF input for doubt solving and summarisation.

7. Conclusion

The code base is a stronger engineering foundation than the original submission: configuration, error handling, schema-validated quizzes, tests and containerisation are all improvements. Fixing the three JavaScript mismatches (C1-C3) and choosing a valid Gemini model (H1) will make the application usable again; rebuilding the interactive quiz (H2) and adding timelines and resources to the learning path (H3) will bring it back in line with the scenarios in the original document.

Submitted by: Varsha